

Identifying LLMs from Their Response

Cmpsci 448 Midterm by Joseph Korsak

Github/Code: <https://github.com/jvkorsak/CMPSCI448-Midterm>

Problem definition and dataset curation

1.1 Task

Given a response y written by an LLM, predict which LLM family of {Claude, GPT, Gemini/Gemma, Llama} generated it. This is a 4-way, single-label text classification problem. Report accuracy and macro-F1. The classes are balanced, so chance accuracy is $1/4 = 25\%$. As a harder secondary task, classify the exact model (12 classes, chance accuracy 8.3%).

1.2 Data Source

Use the public model outputs released with AlpacaEval (Li et al., 2023; github.com/tatsu-lab/alpaca_eval, folder results/<model>/model_outputs.json, commit cd543a1, August 2025). AlpacaEval is an instruction-following benchmark of 805 prompts. Its authors and the people who submitted models ran each model on those prompts with that model's standard chat template and published the responses. I downloaded each model's file directly from that GitHub folder, pinned to the commit above so the data is reproducible, and merged them into one table, data/dataset.csv.

The prompts are public benchmark questions and contain no private or personal data. We chose this source for three reasons: it is public and redistributable, it covers many families, and every model answered exactly the same prompts. The last point gives us a controlled experiment, because any difference between classes can only come from the models themselves. We used 12 models GPT: gpt-3.5-turbo-0613, gpt4_1106_preview, gpt-4o-2024-05-13, Claude: claude-2.1, claude-3-opus-20240229, claude-3-5-sonnet-20240620, Gemini/Gemma: gemini-pro, gemma-2b-it, gemma-7b-it, and Llama: llama-2-70b-chat-hf, Meta-Llama-3-70B-Instruct, Meta Llama-3.1-405B-Instruct-Turbo.

1.3 Cleaning, task labels, and Split

For Cleaning, We dropped the 23 empty responses (22 from gemma-2b-it and 1 from gemini-pro). This leaves 9,636 rows, with 2,392–2,415 per family, so the classes are balanced to

within 1%. We changed nothing else in the text. Markdown, newlines and casing are kept because they are part of each model's style.

For task type. (for a per-task breakdown). AlpacaEval has no task labels, so we assign each prompt a task using ordered keyword rules, which are stored as the task column of dataset.csv: coding (75 prompts), math (38), writing (127), and qa/advice for everything else (565). We spot-checked 24 prompts by hand. The rules were mostly right but noisy: for example, "Create a short summary of this paper" is tagged coding because its abstract contains the word "algorithm".

For Splits, Each prompt has 12 answers. A random row split would put the same question in both train and test, and the model could then learn topics instead of authors. We therefore split by prompt, 70/15/15: 563/120/122 prompts, which is 6,738/1,439/1,459 responses. Every model's answer to a test prompt is unseen during training.

Models and Training

2.1 Text Representation

We use a regex tokenizer (`tokenize()` in `midterm_project.py`) that makes every word, number, punctuation mark, and markdown symbol its own token, and turns each newline into a special `<nl>` token. Casing is kept. This choice is deliberate: an ordinary word tokenizer would throw away `***` `###`, list numbering, and paragraph breaks, and these are exactly the kind of habits that differ between assistants. The vocabulary is built from the training split only and keeps every token seen at least twice (27,927 tokens; unknown tokens map to `<unk>`). Sequences are cut to 400 tokens, which covers the whole response for 65% of samples (the median length is 311 tokens). In the input output setting, the prompt is cut to 120 tokens and joined to the response with a `<sep>` token.

2.2 CNN

We use a Kim (2014)-style convolutional network (TextCNN in `midterm_project.py`). A 128-dimensional embedding layer, trained from scratch; three parallel 1-D convolutions with kernel widths 3, 4 and 5 and 128 filters each, each followed by ReLU. Global max-pooling over time, with padded positions masked out. Dropout 0.5 on the 384 pooled features, then a linear layer to 4 outputs. Each filter acts as a learned n-gram detector, and max-pooling asks whether that pattern occurs anywhere in the response. This matches the intuition that fingerprints are local phrases like "Certainly! Here's" or `***:` The model has 3.77M parameters, of which 3.57M are embeddings.

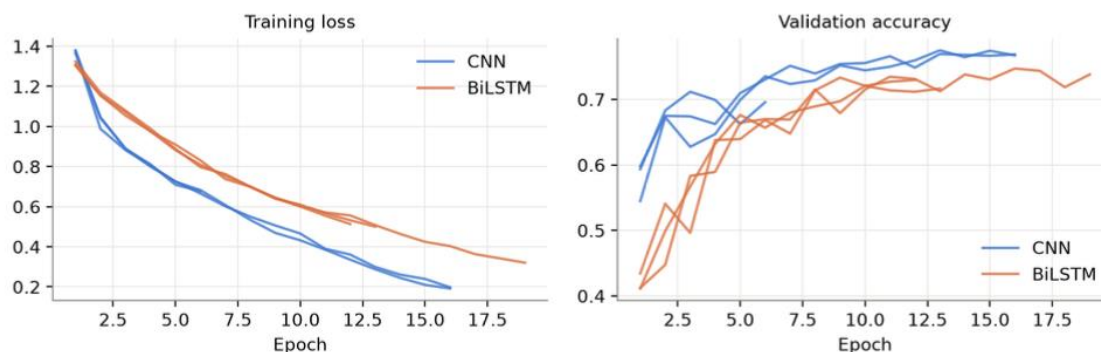
2.3 RNN (bidirectional LSTM)

The RNN (BiLSTM) starts with the same 128-dimensional embedding (dropout 0.5), followed by a 1-layer bidirectional LSTM with 128 hidden units per direction. The 256-dimensional hidden states are pooled over time in two ways: masked mean and masked max. The two results are concatenated into a 512-dimensional vector, passed through dropout 0.5, and fed to a linear layer. The model has 3.84M parameters.

We first used `pack_padded_sequence`, but on our CPU its backward pass was about 20× slower (6 s vs 0.3s per batch in a timing test). Instead, we batch examples of similar length together (length bucketing) and run the LSTM on the padded batch directly. Bucketing leaves only a few padding tokens per batch, and the pooling step masks those out.

2.4 Training Procedure

- Loss: cross-entropy. Optimizer: Adam (learning rate 1e-3, weight decay 1e-5), batch size 64, gradient-norm clipping at 5.
- Up to 20 epochs. After each epoch, we compute validation macro-F1 and keep the best checkpoint.
- Training stops after 3 epochs without improvement; runs lasted 6–20 epochs (Figure 2).
- Each configuration is trained with 3 random seeds (0, 1, 2) on the same split. We report the mean $\pm$ standard deviation on the test split, which each run evaluates only once.
- Reference model: TF-IDF features (word 1–2-grams and character 2–5-grams, case kept, sublinear TF) fed to logistic regression ($C = 10$).
- Hardware: 4 CPU cores, no GPU.



RQ1: Can we identify which LLM generated a Response?

Model (input = LLM_output)	Accuracy	Macro-F1
Majority class / chance	25.1%	–
CNN (3 seeds)	74.5 ± 3.0%	74.5 ± 3.1%
BiLSTM (3 seeds)	74.6 ± 1.8%	74.3 ± 2.0%
TF-IDF + logistic regression	79.0%	78.9%
CNN, 12-way exact model (chance 8.3%)	53.4%	52.5%

Yes, far above chance. Both required models identify the family about three times more often than chance would. The CNN and the BiLSTM are tied on average, but the CNN varies more between seeds (one seed stopped early at epoch 6 with 71.1% accuracy) and trains about twice as fast. The linear TF-IDF model beats both by about 4.5 points. We read this as a data-size effect. With about 6.7k training texts, a 3.8M-parameter network with randomly initialized embeddings has to learn every feature from scratch. TF-IDF starts with hundreds of thousands of ready-made n-gram features, and most of the signal here appears to be shallow and lexical: typical openers and markdown habits. Even the exact model can be recovered 53% of the time, and 46% of its errors are confusions between models of the same family (random errors would land there only 18% of the time).

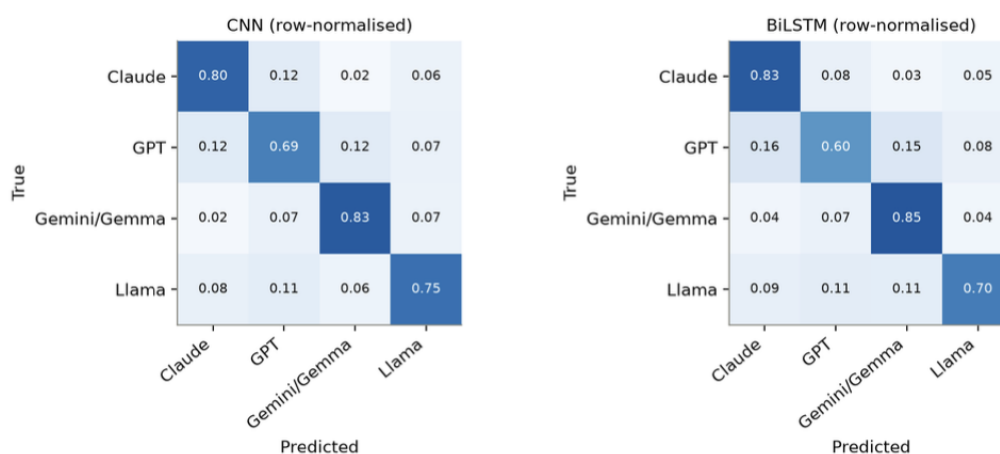
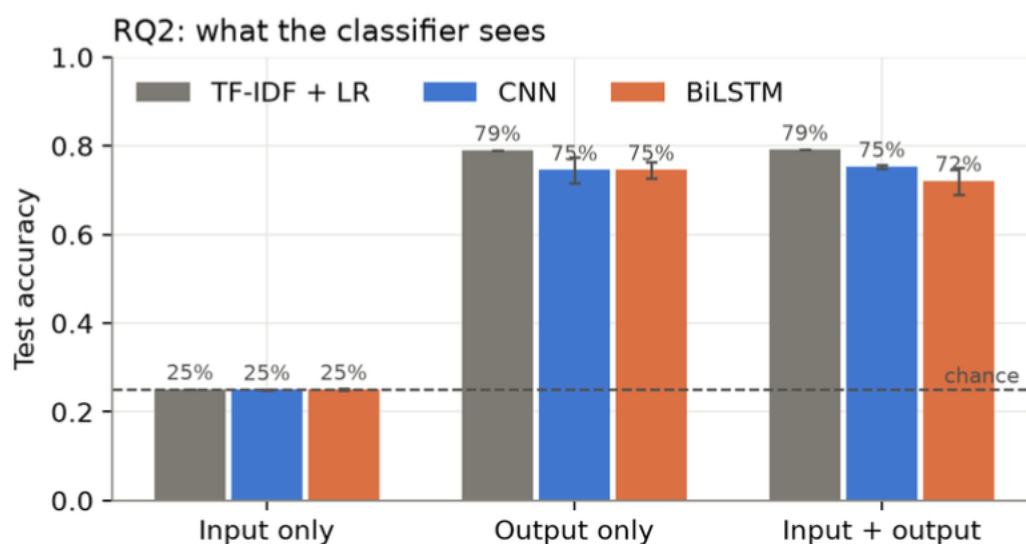


Figure 3. Confusion matrices on the test set (seed 0), normalized by row.

Averaged over the three seeds, Gemini/Gemma is the easiest family (CNN /BiLSTM recall 81% / 88%). Claude (76% / 79%) and Llama (75% / 71%) follow. GPT is the hardest (66%/ 61%) and is misread as Claude or Gemini/Gemma about equally often. A plausible reason is that

GPT's style is the "default assistant" style that other assistants were tuned to imitate. By individual model (CNN, seed 0), the most distinctive are gemma-7b-it (93% recall) and claude-2.1 (90%). The least distinctive are gpt-3.5-turbo (61%) and Claude 3 Opus (63%). So a newer model in a family is not always easier to identify: Claude 3 Opus is much harder than Claude 2.1. By task type (CNN, seed 0), accuracy is highest on open-ended QA and advice prompts (81%) and lowest on writing (66%) and coding (71%). Coding and writing prompts constrain the content more, which leaves less room for style.

RQ2 Do User Prompts Help identify the LLM?



Input	TF-IDF	CNN	BiLSTM
Input (prompt) only	25.1%	25.0 ± 0.1%	25.0 ± 0.2%
Output only	79.0%	74.5 ± 3.0%	74.6 ± 1.8%
Input + output	79.1%	75.3 ± 0.5%	72.1 ± 3.0%

Can the LLM be predicted from the prompt alone?

Not in our dataset, and the reason is structural. Every test prompt appears exactly four times, once for each family. A classifier that sees only the prompt must give the same answer for all four copies, so it is right on exactly one of them. Every prompt-only classifier therefore scores exactly 1/4, whatever it learned. The results match this: all three models sit at 25.0–25.1%. Their macro-F1 is lower (15–21%) because they mostly predict a single class.

(ii) Does including the prompt improve classification?

No. The changes are CNN +0.8, BiLSTM -2.5 and TF-IDF +0.1 points, all within one standard deviation of the variation between seeds. This is expected too: the prompt carries no label information here, so the most it can do is give context for reading the response.

(iii) Why could prompt-only classification work "surprisingly well" elsewhere?

In data collected in the wild, such as chat logs or scraped conversations, different models are used by different people for different jobs. Coders may favor one assistant and students another, and datasets are often gathered at different times or with different prompt templates. The prompt then leaks the label through the collection process, not through anything the model wrote.

To test this, we simulated biased collection (`rq2_sim()` in `midterm_project.py`). Each of the 805 prompts is given to exactly one family. With probability b , the family is chosen by the prompt's source subset: `selfinstruct` and `vicuna` go to GPT, `koala` to Claude, `oasst` to Llama, and `helpful base` to Gemini/Gemma. Otherwise, the family is picked at random. We then train prompt-only and response-only models, with 5 repetitions for each value of b .

Prompt-only accuracy is at chance when there is no bias (26%). It climbs to 29–31% at $b = 0.3$, 41–46% at 0.6, and 57% at 0.9. At $b = 0.9$, the majority-class baseline is only 38%, so the classifier is really reading the prompt. At that point, prompt-only accuracy is almost as high as response-only accuracy on the same data. The lesson is methodological. A high prompt-only score is a warning sign of a shortcut in the dataset, not evidence that prompts carry a model's fingerprint. A shared-prompt design like AlpacaEval's removes this confound by construction.

Limitations and Threats to Validity

- Single-turn benchmark responses only. AlpacaEval responses come from default API settings with no system prompt. Real deployments add system prompts and personas, which can change formatting a lot. Our accuracies should not be read as detection rates in the wild.
- 2023–2024 models. The newest model in the set is from mid-2024, and fingerprints drift between versions (compare Claude 2.1 and Claude 3 Opus in Section 3).
- Four families only. We restricted the study to GPT, Claude, Gemini/Gemma and Llama. Open models such as Qwen and Mistral were probably tuned on data from these assistants, so adding them would likely make the task harder.
- Gemini/Gemma pools a proprietary model with two small open models. The Gemma models may be part of why this family is so easy to identify.

- Noisy task labels. The keyword rules mislabel some prompts, so the per-task breakdown in Section 3 is approximate.
- Limited compute (CPU only). We used no pretrained embeddings or transformers and did not tune hyperparameters beyond sensible defaults. The test set has 122 prompts, and the CNN's seed spread is ± 3 points, so differences smaller than about 3 points should not be over-interpreted.

Lessons and Experience

This project gave me a much better understanding of how text classification actually works beyond just training a model and reporting its accuracy. One of the biggest things I learned was how important the way a dataset is constructed and split can be. Because every LLM answered the same prompts, splitting randomly by response could have allowed the models to recognize the prompt rather than the writing style of the LLM. Splitting by prompt helped prevent this type of data leakage and made the results more meaningful. This was something I would not have thought of. At first, that is how I was planning on doing it, but as I researched the project, it was clear that, with my dataset, I had to change the method.

I also learned that a more complicated model is not always the best model. I originally expected the CNN and BiLSTM to outperform a simpler method, but TF-IDF with logistic regression achieved the highest accuracy at 79%, compared with about 75% for the neural networks. This showed me that much of the information separating these LLMs came from relatively simple patterns such as wording, punctuation, markdown, and common phrases rather than very complex language patterns. It also helped me better understand what CNNs and RNNs are actually learning from text instead of viewing them only as algorithms. This is a core idea of the CNN.

Another important lesson came from the prompt-only experiment. The prompt by itself produced about 25% accuracy, which is essentially chance, and adding the prompt did not meaningfully improve classification. The simulated biased dataset showed me why this mattered, as a classifier can appear successful because of patterns in how data was collected rather than because it learned what we intended it to learn. This made me more aware of shortcuts and confounding variables in ML experiments.

I also used AI as a tool throughout the project, particularly to help me understand unfamiliar concepts, troubleshoot parts of the code, and think through how to interpret results. I found that AI was most useful when I treated it as a starting point rather than automatically accepting its answers. I still needed to check the code, understand why changes worked, and

make sure I could explain the models and results myself. It came in very helpful with debugging, as this was one of my first real times working with PyTorch, pandas, ... and without AI for debugging and finding small syntax errors, it would have taken me much longer to do.

Overall, this project helped me become more comfortable implementing CNNs and RNNs, evaluating models, and thinking critically about whether a model's performance actually supports the conclusion I am trying to make.

